Chat Completions API

(https://github.com/learn-co-curriculum/phase-3-ai-openai-chatcompletion-api) (https://github.com/learn-co-curriculum/phase-3-ai-openai-chatcompletion-api/issues/new)

Learning Goals

- Use the Chat Completion API.
- Describe the key parameters and the response of the API.
- Use the CLI to get user input and display responses.

Introduction

We have set up our environment to connect with the OpenAI models. In this lesson, we will be working in the same `playground-ai` directory to explore the **Chat Completion API** (https://platform.openai.com/docs/guides/gpt/chat-completions-api) by OpenAI.

The Chat Completions API is the latest API for natural language processing models like GPT-4. We will be creating 2 functions and explore the main parameters:

- Basic Prompt Function.
- Context Aware Function.

Before we start, comment out the `try-except` statement you added when testing the local environment. Your `ai.py` file should look like the following:

```
import os
import openai
from dotenv import load_dotenv, find_dotenv

_ = load_dotenv(find_dotenv())
```

```
openai.api_key = os.environ["OPENAI_API_KEY"]

# messages = [{"role": "user", "content": "What is the capital of New York?"}]
# try:
#     response = openai.ChatCompletion.create(
#         model="gpt-3.5-turbo",
#         messages=messages,
#         temperature=0,
#     )

#     print(response.choices[0].message["content"])
# except Exception as e:
#     print(e)
```

This will prevent unnecessary calls to the API when importing functions from the `ai.py` module in other modules.

Basic Prompt Function

We will start with the most basic function that can be used for a single prompt. The user will interact with the function through a CLI interface.

Add the following function definition to your `ai.py` file:

```
def get_completion(prompt, model="gpt-3.5-turbo", temperature=0):
    messages = [{"role": "user", "content": prompt}]

    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=temperature,
    )

    return response.choices[0].message["content"]
```

You can call the function in the `ai.py` file with a prompt and get back a response:

```
prompt = "What is the capital of New York?"
get_completion(prompt) # model and temperature will use default values
```

We will be exploring the following components of this function:

- Function parameters.
- The `messages` list.
- `ChatCompletion` API call.
- Response.

Function Parameters

The `ChatCompletion` API takes in a `[messages` list](https://platform.openai.com/docs/api-reference/chat/create#chat/create-messages (https://platform.openai.com/docs/api-reference/chat/create#chat/create-messages)) (discussed later in this lesson) as its input but we have chosen to simplify our basic function by allowing the caller of the function to only provide a single prompt using the `prompt` parameter.

The `model` parameter allows the caller to specify the OpenAI model that should be used. For example, we can provide `gpt-4` as the value if we want to use the GPT-4 model. We have specified GPT-3.5 Turbo as the default model since it's the most cost effective model.

The `temperature` parameter controls the randomness of the response. It affects the creativity and diversity of the model's output. Its value is a floating-point value that ranges from 0.0 to 1.0.

A lower `temperature` value makes the response more deterministic, i.e., the response is likely to be consistent across function calls. A higher value makes the response more random allowing for a wider range of possible responses. This can lead to novel responses but may also produce less coherent or sensible responses.

Your choice of the temperature value will depend on the desired behavior of your application. If you want to prioritize accuracy and consistency use a lower value (documentation chatbots) and if you want to generate varied responses (idea generation chatbots) use a higher value.

The `messages` List

The `messages` list parameter (**doc reference** (https://platform.openai.com/docs/api-reference/chat/create#chat/create-messages)) to the Chat Completion API represents the conversation between the user and the assistant.

Each object in the list contains at least the `role` and `content` property. The `content` value is the input text or "prompt" to the system.

The `role` property can have the following values:

- `system` : The first message in the list is usually a message with a role of `system` . It sets the behavior or provides context for the assistant.
- `user` : Following the system message, you can include user messages with a role of `user` . These messages represent user input or queries.
- `assistant` : The assistant's previous replies are included as messages with a role of `assistant` .

Here's what a messages list with multiple messages may look like:

```
'messages': [
    {'role': 'system', 'content': '...'},
    {'role': 'user', 'content': '...'},
    {'role': 'assistant', 'content': '...'},
]
```

In our example, we are using the following value:

```
messages = [{"role": "user", "content": prompt}]
```

This instructs the model to expect a user input without any context and provide a response based on the model's default context.

ChatCompletion API Call

Here's what our API call looks like:

```
response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
```

```
        temperature=temperature,
    )
```

The `openai` module has a `ChatCompletion` class that provides a `create` method which makes it easier to make a request to the API model.

We provide the `create` method the model we want to use, the messages list, and the desired temperature of the responses. The exact method call may vary depending on the library you're using. Check out the available OpenAI libraries on the **Libraries doc page (https://platform.openai.com/docs/libraries)** .

Response

In our `get_completion` function, add a print statement right before the return statement to view the response object.

The response object should look like this:

```
{
  "id": "chatcmpl-7aoO5OF9Cq4Mo3mrYXtNg9ddQjVQr",
  "object": "chat.completion",
  "created": 1689007853,
  "model": "gpt-3.5-turbo-0613",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "The capital of New York is Albany."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 15,
    "completion_tokens": 8,
```

```
      "total_tokens": 23
    }
}
```

The two main properties you should be aware of are `choices` and `usage`.

The `choices` list will have the generated response objects. In this case, each object in the `choices` list is an object with a `message` property which holds the response we want to display to the user. We can also use this response in the `messages` list for holding the chat exchange context.

The `usage` object displays the number of tokens the request took. You should be aware of the token usage to set up appropriate usage limits on your account.

Remove any `print` statements from the `get_completion` function before moving on to the next section.

CLI Integration

Now that we have written a `get_completion` function and understand how to work with it, let's look at integrating the function into other parts of our app.

Create a `app.py` file in the `playground-ai` folder and add the following code:

```
from ai import get_completion

prompt = input('Enter prompt: ')

response = get_completion(prompt)

print(response)
```

Run the app by running the following command:

```
pipenv run python app.py
```

This should bring up a user prompt in the terminal. Feel free to experiment with different inputs. Here's an example:

```
# user input
Enter prompt: What is the capital of New York?
# model response
The capital of New York is Albany.
```

Context Aware Function

A context aware function allows you to pass in a list of messages as input. This enables us to keep track of the correspondence between the user and the AI assistant.

Open the `ai.py` file and add the following:

```
def get_completion_from_messages(messages, model="gpt-3.5-turbo", temperature=0):
    response = openai.ChatCompletion.create(
        model=model,
        messages=messages,
        temperature=temperature,
    )
    return response.choices[0].message["content"]
```

Here's what an example usage might look like:

```
messages = [
        {"role": "system", "content": "You are a helpful assistant."},
        {"role": "user", "content": "Who won the world series in 2020?"},
        {"role": "assistant", "content": "The Los Angeles Dodgers won the World Series in 2020."},
        {"role": "user", "content": "Where was it played?"}
    ]

get_completion_from_messages(messages)
```

The content in the message with the `system` role is the default context for GPT models. You could provide a different context depending on the kind of application you're building. You can check out examples in the **GPT Best Practices guide (https://platform.openai.com/docs/guides/gpt-best-practices/tactic-ask-the-model-to-adopt-a-persona)** .

Conclusion

We have learned how the OpenAI ChatCompletion API works and how to integrate our custom functions in our app. There are other endpoints provided by OpenAI but this is the main endpoint for interfacing with their latest GPT models.

Resources

- OpenAI Doc Pages:
  - **Chat Completions API Overview. (https://platform.openai.com/docs/guides/gpt/chat-completions-api)**
  - **Chat Completions API Reference. (https://platform.openai.com/docs/api-reference/chat/create)**
  - **Messages List API Reference. (https://platform.openai.com/docs/api-reference/chat/create#chat/create-messages)**
  - **Libraries. (https://platform.openai.com/docs/libraries)**
  - **GPT Best Practices. (https://platform.openai.com/docs/guides/gpt-best-practices)**